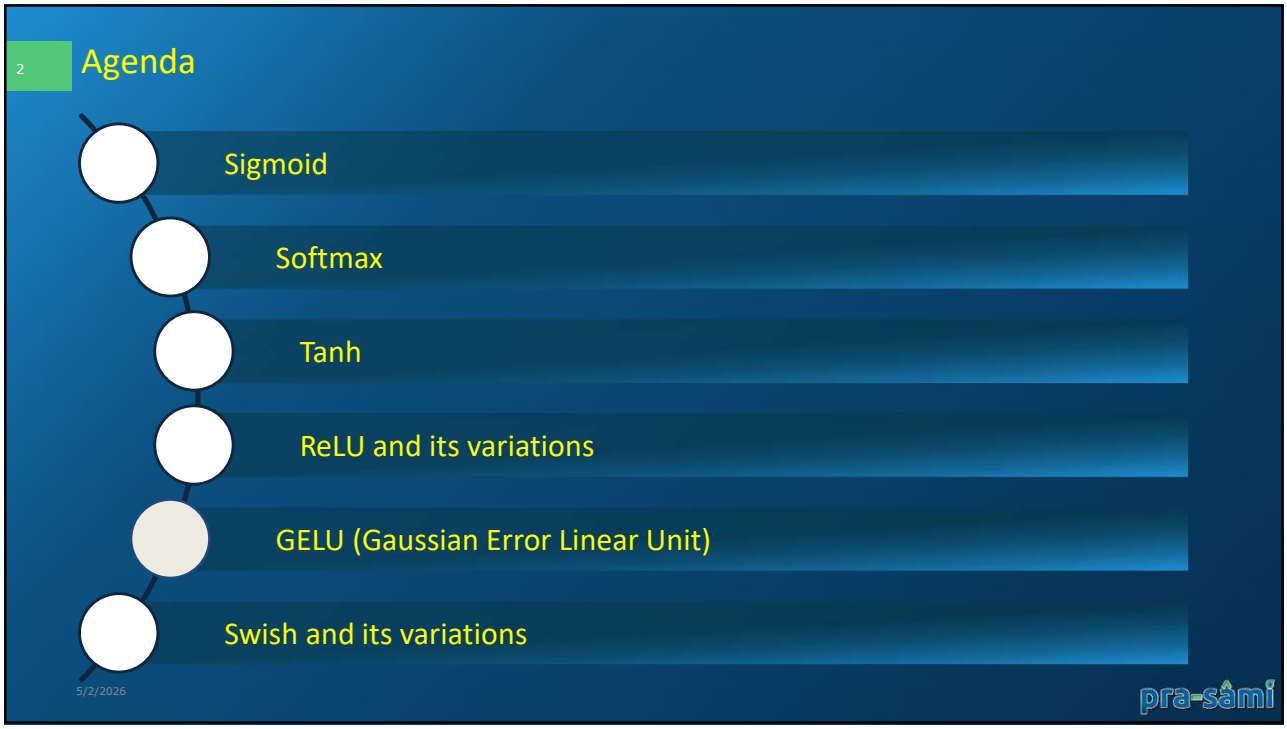
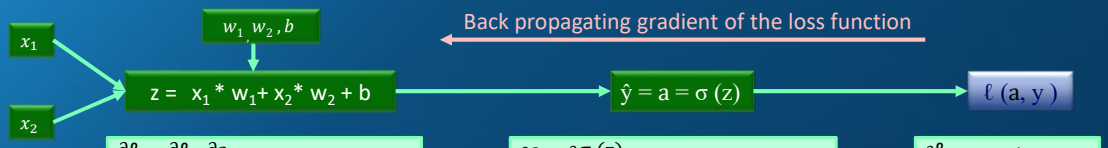

Activation Function
Deep Neural Networks
Session 03
Pramod Sharma
pramod.sharma@prasami.com


2
Agenda
Sigmoid
Softmax
Tanh
ReLU and its variations
GELU (Gaussian Error Linear Unit)
Swish and its variations
5/2/2026
pra-sami

3 Forward and Back Propagation



$$\frac{\partial \ell}{\partial z} = \frac{\partial \ell}{\partial a} \cdot \frac{\partial a}{\partial z}$$
$$= \{ -\frac{y}{a} + \frac{1-y}{1-a} \} * a * (1-a)$$
$$= a - y$$

$$\frac{\partial a}{\partial z} = \frac{\partial \sigma(z)}{\partial z} = \sigma(z) . (1 - \sigma(z))$$
$$\frac{\partial a}{\partial z} = a . (1 - a)$$

$$\frac{\partial \ell}{\partial a} = -\frac{y}{a} + \frac{1-y}{1-a}$$

z = X * W + b

ŷ = a = σ (z)

$\sigma(z) = \frac{1}{1+e^{-z}}$

ℓ(a, y) = - [y * log(a) + (1-y) * log(1-a)]

$\Rightarrow$

$$\frac{\partial \ell}{\partial w1} = x_1 . \frac{\partial \ell}{\partial z} = x_1 . (a-y)$$
$$\frac{\partial \ell}{\partial w2} = x_2 . \frac{\partial \ell}{\partial z} = x_2 . (a-y)$$
$$\frac{\partial \ell}{\partial b} = \frac{\partial \ell}{\partial z} = (a-y)$$

$\Rightarrow$

$$w_1 = w_1 - \alpha * \frac{\partial \ell}{\partial w_1} = w_1 - \alpha * x_1 * (a-y)$$
$$w_2 = w_2 - \alpha * \frac{\partial \ell}{\partial w_2} = w_2 - \alpha * x_2 * (a-y)$$
$$b = b - \alpha * \frac{\partial \ell}{\partial b} = b - \alpha * (a-y)$$

Where α is learning rate. The cost function is

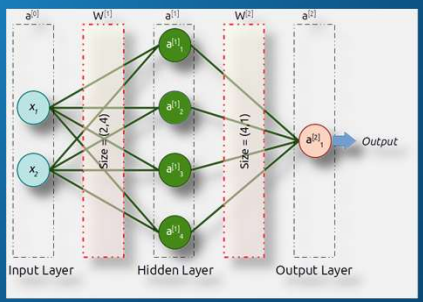
$$J(W, b) = \frac{1}{m} * (\Sigma \ell (a, y))$$

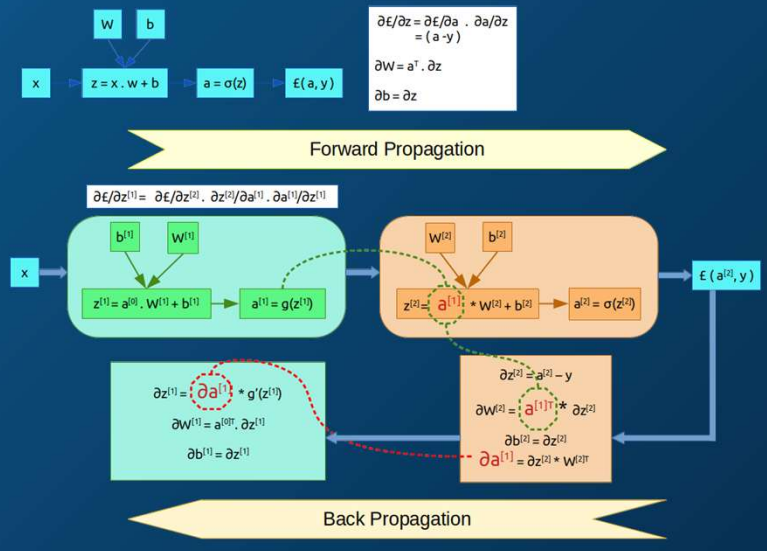
Hence $\frac{\partial J}{\partial w_1} = \frac{1}{m} * (\Sigma \frac{\partial \ell (a, y)}{\partial w_1})$

5/2/2026

pra-sâmi

4 Neural Network





5/2/2026

pra-sâmi

5

Activation Function

pra-sami

6

Activation Functions: The Neural Gatekeepers

- An Activation Function is applied to each Neuron to:
  - Transform weighted inputs into outputs
  - Introduce non-linearity into the network
- How It Works
  - Takes input: $z = xW + b$
  - Produces output: $a = \sigma(z)$
- In the Network, Acts as a gate:
  - Activates ("fires") or suppresses neuron output
  - Controls information flow through layers

- Enables learning of complex (non-linear) patterns
- Affects training stability and gradient flow
- Must be computationally efficient (applied at every neuron)

pra-sami

7

Activation Functions: Beyond Non-Linearity

- Activation functions do more than introduce non-linearity—they also:
  - Control gradient flow
  - Stabilize training
  - Shape output distributions
- Choice directly affects training speed and model stability
- Examples:
  - Sigmoid/Tanh → smooth output, but suffers from vanishing gradients
  - ReLU → fast convergence, but risk of dying neurons
  - Leaky ReLU / Swish / GELU → Attempt to address ReLU limitations ; Often perform better in specific domains (e.g., NLP)
- ReLU is the default due to:
  - Simplicity
  - Computational efficiency
  - Strong empirical performance

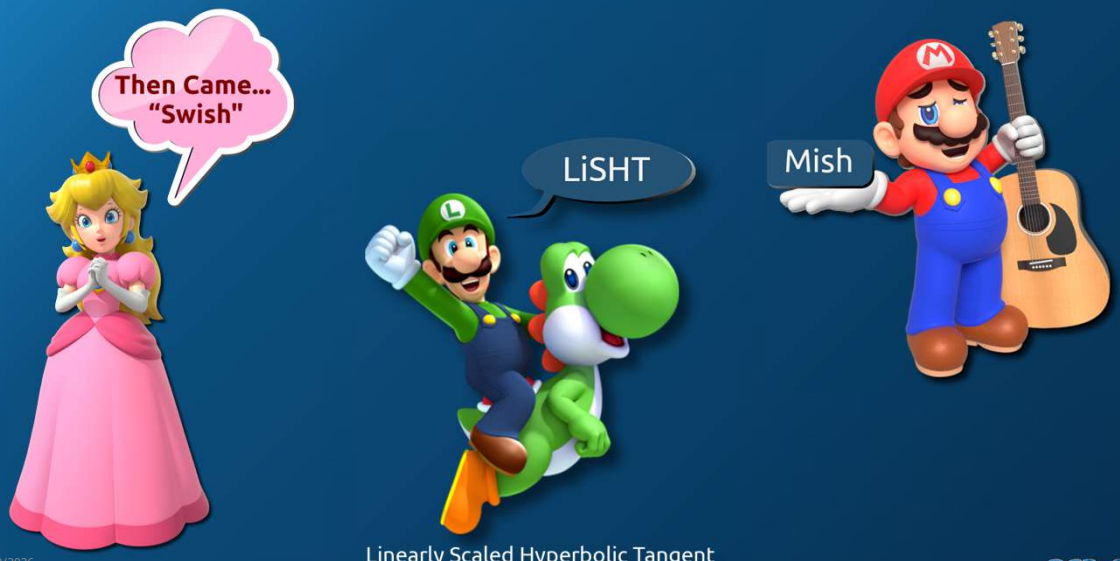
No activation function is universally best
selection depends on the task and architecture.

5/2/2026

pra-sâmi

8

ReLU: The Contenders – Swish, LiSHT & Mish



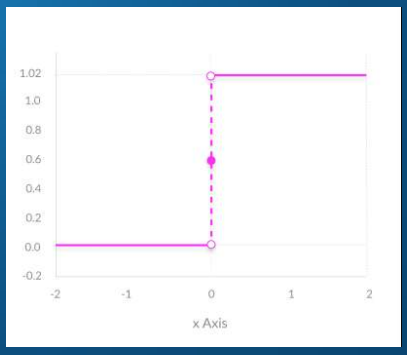
5/2/2026

pra-sâmi

9

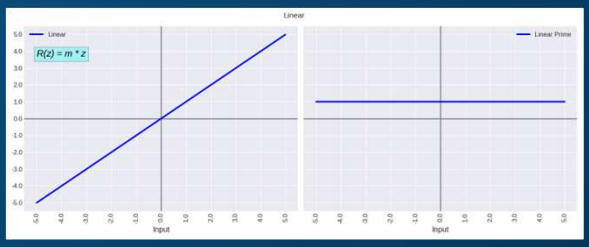
Activation Functions

- Binary Step function



- We already seen this in previous session!!!

- Linear Activation Function



- That will be simple linear regression!
  - There may still be some use cases…

5/2/2026

pra-sami

10

Non-Linear Activation Functions

- There are many popular activation functions

Sigmoid / Logistic | Softmax | Argmax | Tanh

ReLU | Leaky ReLU | Parametric ReLU | ELU (Exponential Linear Unit)

GELU (Gaussian Error Linear Unit) | Swish/H-Swish | LiSHT | Mish

- Stay tuned… it's an active research area…

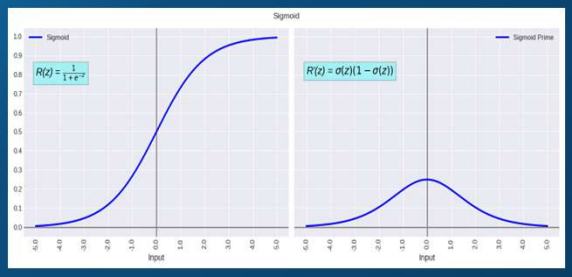
After Mish and LiSHT, newer activations exist,
but GELU, Swish, and ReLU variants remain the industry standard.

5/2/2026

pra-sami

11

Sigmoid



- Takes a real-valued input and outputs a value between 0 and 1, i.e., [0,1].
- Easy to work with and widely used as an activation function in output layer
- Non-linear, continuously differentiable, monotonic, with a fixed output range.
- Well-suited for binary classification tasks.

pra-sami

12

Sigmoid: Connection to Log-Odds (Logistic Function)

- Odds Definition
  - Odds of an event: Odds= $\frac{p}{1-p}$
  - Example: If p=0.75, odds = 0.75/0.25 = 3 (3:1 in favor)
- Log-Odds (Logit Function)
  - logit(p) = $\ln(\frac{p}{1-p})$
  - Range: $(-\infty,+\infty)$ when $p \in (0,1)$
- Linear model predicts log-odds, not probability directly:
  - $\ln\left(\frac{p}{1-p}\right) = X.W + b$
- Solving for Probability
  - Exponentiate both sides: $\frac{p}{1-p} = e^{X.W+b}$
  - Solve for p:

$p = \frac{e^{X.W+b}}{1+e^{X.W+b}}$

$= \frac{1}{1+e^{-(X.W+b)}}$

$= \frac{1}{1+e^{-z}}$

- This is exactly the sigmoid function!

pra-sami

13 Sigmoid – Drawbacks

- Gradient Saturation
  - For very large or very small inputs, sigmoid outputs approach 0 or 1
  - Gradients become very small (near zero) → slow or stalled learning
- Vanishing Gradient Problem
  - During backpropagation, repeated multiplication of small gradients
    - Causes gradients to shrink exponentially in deep networks
    - Early layers learn very slowly or not at all
- Non-Zero-Centered Output
  - Output range: [0, 1]
  - Always positive, not centered around zero
    - Biased gradient updates
    - Slower convergence during optimization
- Input Scaling Sensitivity
  - Large input values push sigmoid into saturation regions
  - Inputs are often normalized/scaled to:
    - Keep activations in the sensitive (learning) region
    - Improve training speed

14 Softmax – Origin & Purpose

- Rooted in statistical physics (Boltzmann/Gibbs distribution)
- Introduced by Ludwig Boltzmann (1868)
- Later applied to decision theory by Robert Duncan Luce (1959)
- Converts a vector of real-valued scores into a probability distribution

Transforms raw model outputs (logits) into interpretable probabilities

15 Softmax – Properties & Mathematical Behavior

- Input: Any real values (positive or negative)
- Output:
  - Each value in (0, 1)
  - Sum of all outputs = 1
- Key Properties
  - Differentiable → suitable for backpropagation
  - Non-linear → enables complex decision boundaries
  - Scale-sensitive → larger inputs dominate probabilities

pra-sami

16 Softmax – Role

- Connection to Logistic Regression
  - Softmax → multi-class generalization of logistic regression
  - Sigmoid → special case of Softmax (for 2 classes)
- In Neural Networks
  - Applied to output layer (logits → probabilities)
  - Works with cross-entropy loss

Like Sigmoid Activation function, Vanishing Gradient is still a problem!

pra-sami

17

Softmax vs. Sigmoid

- Sigmoid

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

- Softmax

$$S(\vec{Z}) = \frac{e^{z_i}}{\sum_{j=1}^{K} e^{z_i}}$$

- For single class value will be [0, x],

$$S(\vec{Z}) = \frac{e^{z_i}}{\sum_{j=1}^{K} e^{z_i}}$$

$$S(\vec{Z}) = \frac{e^{z_1}}{e^{z_1} + e^{z_2}}$$

$$S(X) = \frac{e^{x}}{e^{0} + e^{x}}$$

$$S(X) = \frac{e^{x}}{1 + e^{x}}$$

$$S(X) = \frac{1}{1 + e^{-x}}$$

pra-sami

18

Softmax vs. Argmax

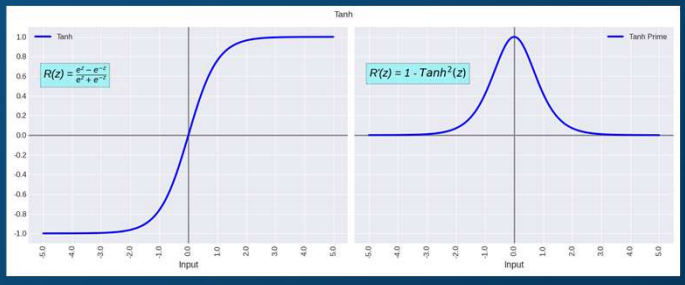
- Both work the same way, Softmax is expected to be a differentiable alternative to argmax
- Argmax returns index of highest value and no idea about other values.
- It is common to train using the Softmax

pra-sami

19

Tanh

- A scaled and shifted version of Sigmoid
- Maps inputs to range: [-1, 1]
- Commonly used in hidden layers
- Helps stabilize activations across layers



- Key Characteristics
  - Non-linear → enables learning complex patterns
  - Zero-centered → outputs balanced around 0
    - Helps faster and more stable optimization
  - Has stronger gradients than Sigmoid near zero
    - Leads to faster learning (in early training stages)

pra-sami

20

Tanh

- Advantages
  - Zero-centered output → balanced gradient updates
  - Handles negative, zero, and positive inputs effectively
  - Faster convergence than Sigmoid (due to steeper gradients near 0)
  - Fully differentiable → suitable for backpropagation
- Limitations
  - Vanishing Gradient Problem - For large inputs:
    - Outputs saturate near -1 or 1
    - Gradients → close to zero
  - Less effective in deep networks compared to ReLU

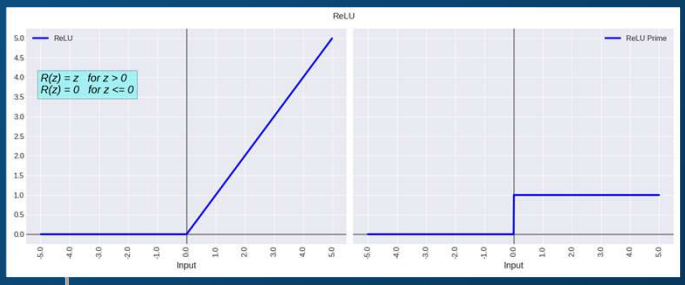
Tanh improves over Sigmoid with zero-centered outputs, but still suffers from vanishing gradients.

- Tanh is generally better than Sigmoid, but often replaced by ReLU in deep networks.

pra-sami

21 Rectified Linear Units (ReLU)

- Defined as:
  - ReLU (x) = max(0, x)
- Non-linear (piecewise linear) activation function
- Output range: [0, ∞)
- Very computationally efficient (simple max operation)
- Produces sparse activations (many zeros)



- Why ReLU is Popular
  - Faster training than Sigmoid/Tanh
  - Helps mitigate the vanishing gradient problem
  - Works well in deep networks
- Usage
  - Commonly used in hidden layers , not in output layer

5/2/2026

pra-sami

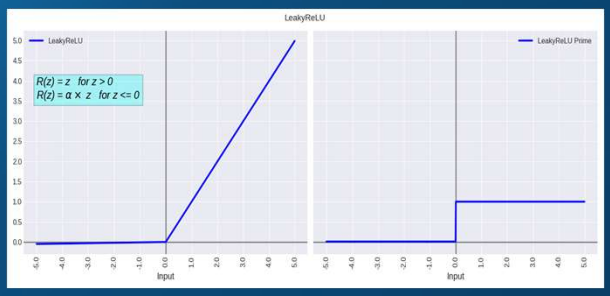
22 ReLU – Limitations

- Dying ReLU Problem
  - For $x < 0$: Output = 0
  - Weights stop updating
  - Neuron becomes inactive (“dies”)
  - Stops responding to input changes
- Unbounded Output
  - Range: [0, ∞); Can lead to:
  - Very large activations
  - Potential instability if not controlled
- Fragile Gradients
  - Some neurons may permanently fall into inactive region
  - Especially with high learning rates or poor initialization

5/2/2026

pra-sami

23 Leaky ReLU



- A variant of the ReLU ; attempt to fix the "dying ReLU" problem
- Allows a small, non-zero gradient for negative values.
  - $R(z_i) = \begin{cases} z_i & if\, z_i \geq 0 \\ a_i \,.\, z_i & if\, z_i < 0 \end{cases}$
  - α is a small constant e.g. 0.01

pra-sami

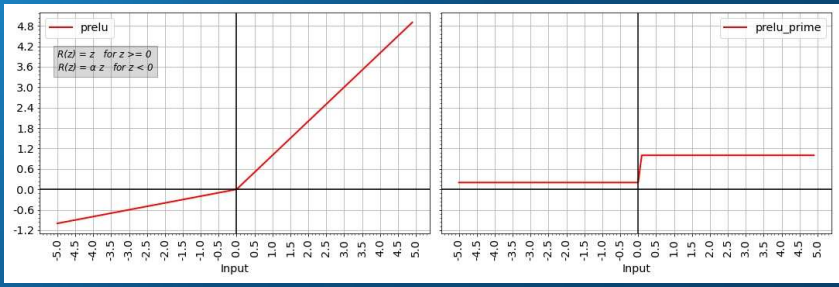
24 Leaky ReLU

Advantages	Limitations
Prevents dying neurons ❖ Non-zero gradient for $z < 0$	Small negative gradient ❖ Learning still slow for $z < 0$
Enables better gradient flow than ReLU	Hyperparameter (α) needs tuning
Simple and computationally efficient	Not zero-centered (like ReLU)
Often leads to more stable training	Can still produce large activations ❖ Unbounded above
	Improvements over ReLU are often incremental

pra-sami

25

Parameterized ReLU (PReLU)



$$\text{PReLU}(z_i) = \begin{cases} z_i & if\ z_i \geq 0 \\ \alpha * z_i & if\ z_i < 0 \end{cases}$$

α is learned during training.

- Generalizes the traditional ReLU with a slope for negative values.
  - The negative slope is learned from data
  - Unlike Leaky ReLU (fixed slope), PReLU adapts during training
- Key Idea: Different layers may require different degrees of non-linearity
  - α is:
    - Initialized (e.g., small value like 0.01)
    - Updated via backpropagation

Recall: α is tuned in ELU.

5/2/2026

pra-sami

26

Parameterized ReLU (PReLU)

- Early layers:
  - Tend to learn larger α
  - More linear behavior → preserves information
- Deeper layers:
  - Tend to learn smaller α
  - More non-linear / selective → improves discrimination

- Advantages
  - Avoids dying ReLU problem
  - Learns optimal activation shape automatically
  - Can improve performance in deep CNNs
- Limitations
  - Adds extra parameters → slight computational overhead
  - Risk of overfitting (especially in small datasets)
  - Gains over ReLU are often task-dependent

PReLU extends ReLU by making the negative slope **trainable**, allowing adaptive non-linearity across layers.

5/2/2026

pra-sami

27

Challenges with ReLU

- Zero Gradient for Negative Inputs
  - ReLU(X) = max(0, x)
  - Neurons in this region do not learn
- Dying ReLU Problem
  - If a neuron consistently receives negative inputs:
    - Weights stop updating (gradient = 0)
    - Neuron becomes permanently inactive
  - Can lead to a significant fraction of dead neurons
- Sensitivity to Initialization & Learning Rate
  - Poor initialization or large updates (e.g., in SGD/optimizers)
  - Can push neurons into the inactive region permanently
- Unbounded Activations
  - Output range: [0, ∞)
  - May cause:
    - Very large activations
    - Potential training instability (if not controlled)
- Partial Remedies: Leaky ReLU / PReLU:
  - Introduce small gradient for $x < 0$
  - Reduce—but do not fully eliminate—the problem

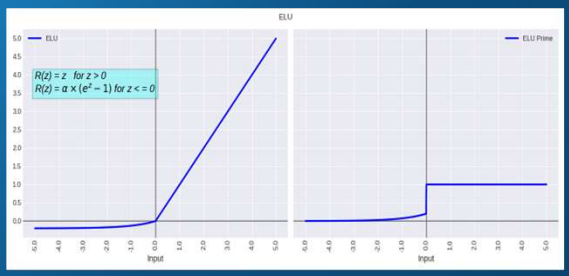
The main limitation of ReLU is the **risk of dead neurons due to zero gradients for negative inputs**.

5/2/2026

pra-sami

28

Exponential Linear Unit (ELU)



$$\text{ELU}(z_i) = \begin{cases} z_i & if\, z_i \geq 0 \\ \alpha * (e^{z_i} - 1) & if\, z_i < 0 \end{cases}$$

α is generally 1.0 but can be tuned.

- Converges faster ; Has alpha constant which should be positive number
- ELU is a strong alternative to ReLU.
  - Provides a smooth curve for negative values
- Unlike to ReLU, ELU is zero-centered, as it outputs negative values for negative inputs and positive values for positive inputs, helping improve gradient flow
- For x > 0, it can blow up the activation with the output range of [0, ∞].

5/2/2026

pra-sami

29

Exponential Linear Unit (ELU)

Advantages

- Prevents dying neurons
  - Allows a small negative output (instead of zero)
- Zero-centered
  - Can lead to faster convergence
- Smoothness
  - ELU is continuous and differentiable everywhere
- Better representation for negative values
  - Network can learn better representations for data with negative correlations.

Disadvantages

- Computational cost
  - Requires computing an exponential function
- Potential for exploding gradients
  - Uses an exponential function for negative inputs. For very large negative inputs, Gradient becomes very small (vanishes)
- Hyperparameter tuning
- Slower convergence
  - It may not always converge as quickly as other activation functions, like swish or SELU
- Limited performance in certain architectures

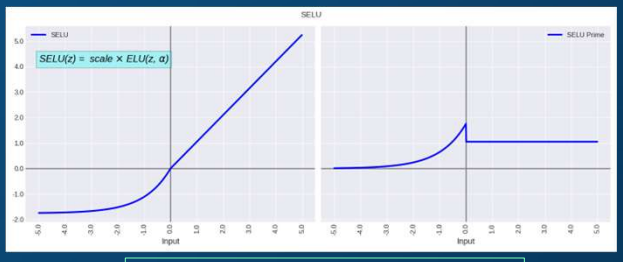
5/2/2026

pra-sami

30

Scaled ELU (SELU)

- Introduced by Klambauer et al. (2017)
- Designed for self-normalizing neural networks
- Key Idea: Self-Normalization
  - With proper conditions (initialization + architecture):
    - Activations become approximately zero-centered
    - Maintain unit variance (std ≈ 1) across layers
  - This stabilizes signal propagation through deep networks
- Helps reduce:
  - Vanishing gradients
  - Exploding gradients
  - Enables more stable and deeper networks



$$\text{SELU}(z) = \lambda * \begin{cases} z, & z > 0 \\ \alpha(e^z - 1), & z \leq 0 \end{cases}$$

Constants (Important)

$\alpha \approx 1.6733$

$\lambda \approx 1.0507$

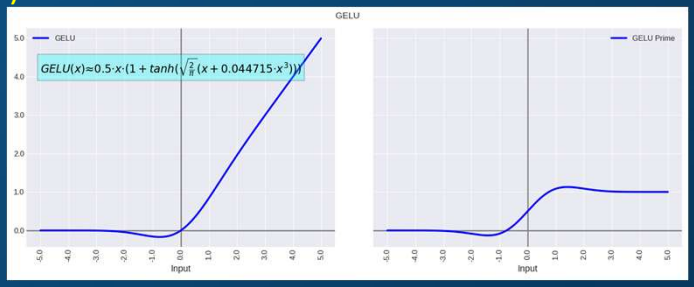
SELU works best when used with:
- Specific weight initialization (LeCun normal)
- AlphaDropout (not standard dropout)
- Consistent use across layers

5/2/2026

pra-sami

31

Gaussian Error Linear Unit (GELU)



- A smooth, non-linear activation function
- Widely used in transformer models (BERT, GPT)
- Core Idea:
  - Instead of hard thresholding (like ReLU), GELU weights inputs probabilistically
  - Scales inputs based on their magnitude and probability
- Contrary to the ReLU, GELU weights its inputs by their value instead of thresholding them by their sign
- GELU (x) = x∗ Φ(x),
  - where Φ(x) : the standard Gaussian cumulative distribution function – refer scipy's norm.cdf(x)
  - Approximately Φ(x) ≈ 0.5 x [1 +tanh{$\sqrt{2/\pi}$ (X + 0.044715 x^3) }]
  - Or x σ (1.702 x)

5/2/2026

pra-sami

32

GELU – Behavior & Trade-offs

- Smooth handling of negative values
  - Unlike ReLU, negatives are not fully discarded
- Improved gradient flow
- Better suited for deep and complex architectures
- Intuition
  - Acts like a soft ReLU, where inputs are gradually activated rather than abruptly switched

- Advantages
  - Strong performance in transformers and NLP models
  - Stable gradients
  - Captures subtle patterns better than ReLU
- Limitations
  - Computationally more expensive than ReLU
  - Slightly slower training
  - Gains are often architecture-dependent
  - Rare risk of large activations if not controlled

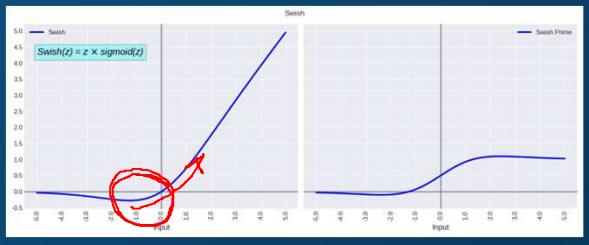
GELU is a smooth, probabilistic version of ReLU, widely used in transformer architectures.

5/2/2026

pra-sami

33 Swish

- Proposed by Google Brain
- Defined as : f(x) = x · sigmoid(x)
- Smooth and non-linear
- Non-monotonic (unlike ReLU)
- Unbounded above, bounded below
- Swish scales inputs smoothly instead of hard-thresholding (like ReLU)
  - Negative values are not completely suppressed
- Enables richer feature learning
- Often matches or outperforms ReLU in deep networks
  - Easy drop-in replacement for ReLU



Hard Swish (H-Swish)

- Used in MobileNetV3
- Approximation of Swish:
  - $f(z) = x \cdot \frac{ReLU6(x+3)}{6}$
  - Where ReLU6(z)=min(max(0,z),6)
- Why important:
  - Much faster on mobile/edge devices
  - Nearly same performance as Swish

5/2/2026

pra-sami

34 Swish – Performance & Trade-offs

- Unbounded above and bounded below
  - Non-monotonic attribute that actually creates the difference
- Can train deeper Swish networks than ReLU networks when using BatchNorm (Ioffe & Szegedy, 2015) despite having gradient squishing property
- With MNIST data set, Swish outperforms ReLU significantly in the range between 40 and 50 layers
  - Both activation functions achieve similar performances up to 40 layers
- In very deep networks, Swish achieves higher test accuracy than ReLU.
- Swish outperforms ReLU on every batch size, suggesting that the performance difference between the two activation functions remains even when varying the batch size.
- Gradient descent problem was still there may be to a lesser degree!

5/2/2026

pra-sami

35

Swish – Performance & Trade-offs

- Empirical Performance
  - Improves accuracy in deep models (e.g., ImageNet benchmarks)
  - Gains become more visible in deeper architectures
  - Performance comparable to ReLU in shallower networks
- Advantages
  - Smooth gradients → better optimization
  - Non-monotonicity → captures complex patterns
  - Works effectively with Batch Normalization
- Limitations
  - Computationally more expensive than ReLU
  - May still exhibit gradient shrinkage (less severe than sigmoid/tanh)
  - Benefits are model- and task-dependent

Swish is a smooth, self-gated activation function that often outperforms ReLU in deeper models

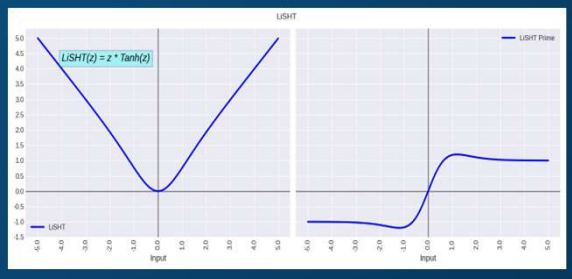
5/2/2026

pra-sami

36

LiSHT (Linearly Scaled Hyperbolic Tangent) Activation Function

- Combines linear and non-linear behavior:
  - f(x)=x·tanh(x)
- Scales the tanh activation with the input itself to enhance gradient flow
- Retains smooth non-linearity (from tanh)
- Adds linear scaling (from x)
- Smooth and non-linear
- Symmetric and zero-centered
- Unbounded (both positive and negative sides)



5/2/2026

pra-sami

37

LiSHT – Performance & Trade-offs

- Helps mitigate:
  - Vanishing gradients (better gradient flow than tanh)
  - Dying neuron problem (unlike ReLU)
- Performance Insight
  - Reported to outperform Swish in some experimental settings
  - Particularly effective in deep architectures (as per original paper)
- Limitations
  - Less widely adopted → limited benchmarking across domains
  - Slightly higher computational cost than ReLU
  - Gains are task-dependent

LiSHT enhances tanh by scaling it with input, enabling better gradient flow and stable deep learning.

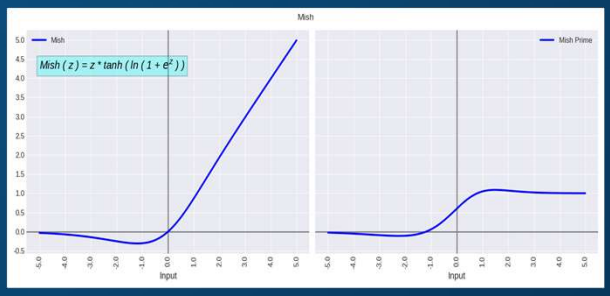
5/2/2026

pra-sami

38

Mish

- A smooth, non-linear, non-monotonic activation function
- Inspired by Swish and designed through systematic experimentation
- Mathematical Form
  - f(z) = z * tanh (softplus (z))
    = z * tanh (ln (1 + e^z))



- Combines smoothness and self-regularization to improve gradient flow
- Uses softplus [ln(1 + e^x)] & tanh
- Produces a smooth transition for both positive and negative inputs
  - Unbounded above, softly bounded below
  - Self-regularizing behavior

5/2/2026

pra-sami

39

Mish – Performance & Trade-offs

- Performance Insights
  - Often outperforms Swish and ReLU in:
    - Computer vision tasks
    - Deep architectures – Especially effective in very deep networks
- Limitations
  - Higher computational cost than ReLU
  - Not yet universally adopted
  - Gains are architecture- and dataset-dependent

Mish is a smooth, self-regularized activation function that often outperforms Swish in deep models —but is not yet a universal default.

5/2/2026

pra-sami

40

Activation Functions: Evolution Map

- Early Activations (Saturating Functions)
  - Focus: Basic non-linearity, probabilistic interpretation
  - Sigmoid
    - Range: (0, 1)
    - Vanishing gradients, not zero-centered
  - Tanh
    - Range: (-1, 1)
    - Zero-centered
    - Still suffers from vanishing gradients
- Breakthrough: ReLU Family
  - Focus: Efficient training + solving vanishing gradients
  - ReLU
    - Fast, simple, effective
    - ❌ Dying neuron problem
  - Leaky ReLU
    - Fixes zero-gradient issue (small slope for $x < 0$)
  - PReLU
    - Learns slope dynamically
    - ❌ Risk of overfitting
- Self-Normalizing & Stabilizing
  - Focus: Stable deep networks
  - SELU
    - Self-normalizing (mean ≈ 0, variance ≈ 1)
    - Requires specific setup (initialization, AlphaDropout)
- Smooth & Probabilistic (Modern Era)
  - Focus: Better gradient flow + richer representations
  - GELU $[x \cdot \Phi(x)]$
    - Probabilistic interpretation
    - Widely used in Transformers (BERT, GPT)
  - Swish $(x \cdot \sigma(x))$
    - Smooth, non-monotonic
    - Outperforms ReLU in deep models
  - Mish $[x \cdot \tanh(\ln(1 + e^x))$
    - Strong smoothness + self-regularization
    - Often better than Swish (task-dependent)
- Hybrid / Advanced Variants
  - Focus: Improved gradient propagation
  - LiSHT $[x \cdot \tanh(x)]$
    - Combines linear + tanh
    - Improves gradient flow

5/2/2026

pra-sami

41

Reflect…

- Which of the following activation functions is zero-centered and helps reduce bias in gradient updates?
  - A) Sigmoid
  - B) ReLU
  - C) Tanh
  - D) Softmax
- Answer: C) Tanh
- Why does ReLU suffer from the "dying neuron" problem?
  - A) It produces very large outputs
  - B) It is not differentiable at zero
  - C) Its gradient is zero for negative inputs
  - D) It saturates at both extremes
- Answer: C) Its gradient is zero for negative inputs
- What is the key difference between GELU and ReLU?
  - A) GELU is linear, ReLU is non-linear
  - B) GELU applies a hard threshold, ReLU is smooth
  - C) GELU weights inputs probabilistically, ReLU thresholds by sign
  - D) GELU outputs binary values
- Answer: C) GELU weights inputs probabilistically, ReLU thresholds by sign
- Which property of Softmax makes it suitable for multi-class classification?
  - A) Outputs are binary
  - B) Outputs are zero-centered
  - C) Outputs sum to 1 forming a probability distribution
  - D) Outputs are unbounded
- Answer: C) Outputs sum to 1 forming a probability distribution

pra-sami

42

Reflect…

- What is a distinguishing characteristic of Mish compared to Swish?
  - A) Mish is linear for positive values
  - B) Mish uses tanh and softplus internally
  - C) Mish outputs binary values
  - D) Mish is monotonic
- Answer: B) Mish uses tanh and softplus internally
- Which activation function is most likely to preserve gradient flow in very deep networks?
  - A) Sigmoid
  - B) Tanh
  - C) ReLU
  - D) Step Function
- Answer: C) ReLU
- Which activation function is most appropriate for multi-class classification with mutually exclusive classes?
  - A) Sigmoid
  - B) Tanh
  - C) ReLU
  - D) Softmax
- Answer: D) Softmax
- Why can non-monotonic activation functions (like Swish/Mish) improve performance?
  - A) They reduce model size
  - B) They enforce linear decision boundaries
  - C) They allow richer feature representations and gradient flow
  - D) They eliminate the need for backpropagation
- Answer: C) They allow richer feature representations and gradient flow

pra-sami

43 Reflect…

- Which activation function is most prone to vanishing gradients in deep networks?
  - A) ReLU
  - B) Leaky ReLU
  - C) Sigmoid
  - D) Swish
- Answer: C) Sigmoid
- What is the primary advantage of Leaky ReLU over ReLU?
  - A) It reduces computational complexity
  - B) It introduces non-linearity
  - C) It allows a small gradient for negative inputs
  - D) It ensures outputs are bounded
- Answer: C) It allows a small gradient for negative inputs
- Which of the following is required for SELU to maintain self-normalizing properties?
  - A) Batch Normalization
  - B) Xavier Initialization
  - C) LeCun Normal Initialization and Alpha Dropout
  - D) Dropout with zero masking
- Answer: C) LeCun Normal Initialization and Alpha Dropout
- Which statement best describes the Swish activation function?
  - A) It is monotonic and bounded
  - B) It is non-monotonic and smooth
  - C) It is piecewise linear
  - D) It outputs only positive values
- Answer: B) It is non-monotonic and smooth

5/2/2026

pra-sami

44 Next Session…

- Loss / Cost Optimization
- Stochastic Gradient Descent (SGD) and others
- Momentum Learning Rates
- Adaptive Learning Rates

5/2/2026

pra-sami

45



5/2/2026

pra-sami